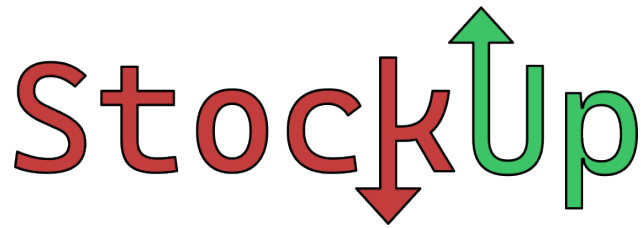


CS 3600 Project Final Report

Jake Gendreau, Michael Crapse

May 5, 2026



Contents

1 Project	3
2 System Design	3
2.1 Database	3
2.2 Backend	3
2.3 Frontend	3
2.4 Stock Predictor	4
3 Functional Components	4
3.1 Database	4
3.2 Backend	4
3.3 Frontend	4
3.4 User Login	4
3.5 Stock Predictor	4
3.6 Leaderboards	4
3.7 Team View	4
3.8 Administration Utilities	5
4 Required Queries	5
4.1 Co-Author Network	5
4.2 H-Index Filter	5
4.3 Q1 Journal Influence Network	6

5	Deviations From the Original Plan	6
6	Progress Reports	7
6.1	Phase I	7
6.2	Phase II	7
6.3	Phase III	7
6.4	Phase IV (Final Phase)	7
7	Validation	8
8	Timeline of Deliverables	12
9	License	12

1 Project

StockUp is a social and financial app which is a combination of fantasy football and real-world investment. On StockUp, users will be able to join a team, invest in stocks, and earn money for their team.

2 System Design

2.1 Database

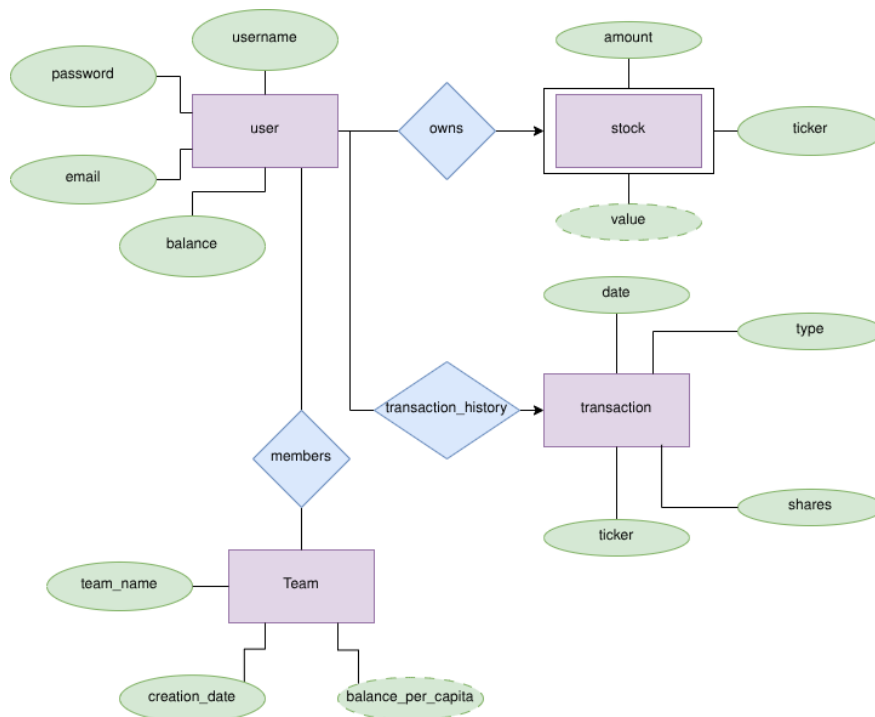


Figure 1: ER Diagram

Django and SQLite3 are used to create and query the database. The tables are generated by Django with the models.py file. Any changes made to the database model are applied by executing the migrate command with manage.py.

2.2 Backend

The backend routes are routed by Django. All routes are implemented in the views.py file. The routing URLs are assigned in the urls.py file from implementations in the views.py file.

2.3 Frontend

The frontend is designed with HTML and Bootstrap templates generated with Django. Templates are stored in the templates directory with static resources from the static directory. Essentially, HTML files are stored in the templates directory and CSS/JS files are stored in the static directory for sharing.

2.4 Stock Predictor

The stock information is gained from the Yahoo Finance package, `yfinance`. The stock predictor model is implemented with `XGBoost` and stored as a file that is restored with the `joblib` package. The `joblib` package allows for the model to be trained once to reduce prediction times.

A prediction may be obtained through a POST request sent to the `invest` route. The POST request body requires a ticker symbol and a sector.

3 Functional Components

3.1 Database

The database is fully implemented using Django's ORM with `SQLite3`.

3.2 Backend

The core features of the backend are implemented with routes, database, and pages. The backend is hosted by Django.

3.3 Frontend

User interface is complete and the content is implemented. The current interface is clean and modern. However, additional refining is desired for improved ergonomics.

3.4 User Login

The user login and sign up forms allow a user to sign into their account or create a new account with their email address and desired password. Additionally, users cannot access pages until they log in.

3.5 Stock Predictor

The stock predictor is partially implemented. The user may search a particular stock, and select the stock industry to view the closing price history of the stock over the course of the past two-hundred days. Once the information is submitted, a graph appears that shows the stock price history along with the 30 day price prediction to help the user to gauge whether the stock is worth purchasing. After a decision is made by the user and the invest option is selected, the user is charged and the stock becomes owned by the user's `StockUp` account.

The stock predictor uses an `XGBoost` Model to predict the stocks. As of May 5, 2026, it is 80% directionally accurate (stock increases / decreases), and can estimate the value of a stock 30 days out to an accuracy of \$20.

3.6 Leaderboards

The leaderboard page shows the top ranking teams based on dollars per capita. On this page, users can view the top 10 stocks held by volume of the top performing teams. They can also view the member count, dollars per capita, and a graph view of the teams.

3.7 Team View

When viewing the details of a team, the user is presented with the team's number of members, balance per capita, and total balance. Additionally, a graph shows each member of the team displayed as a node with name that is connected to the team node. The user may request to join the team by selecting the join team option.

3.8 Administration Utilities

There are administration tools that allow the server administrators to view and modify database values and tables. The utilities are protected with an admin login that requires user privileges to access. These utilities allow for easy debugging and error tracking to resolve any potential issues.

4 Required Queries

4.1 Co-Author Network

Our equivalent to the required Co-author network required query is in the team view page, where a user can view statistics about a team. These statistics include team size, team balance per capita, and team total balance. Additionally, on this page, the user can view a graph of all users who are members of the team.

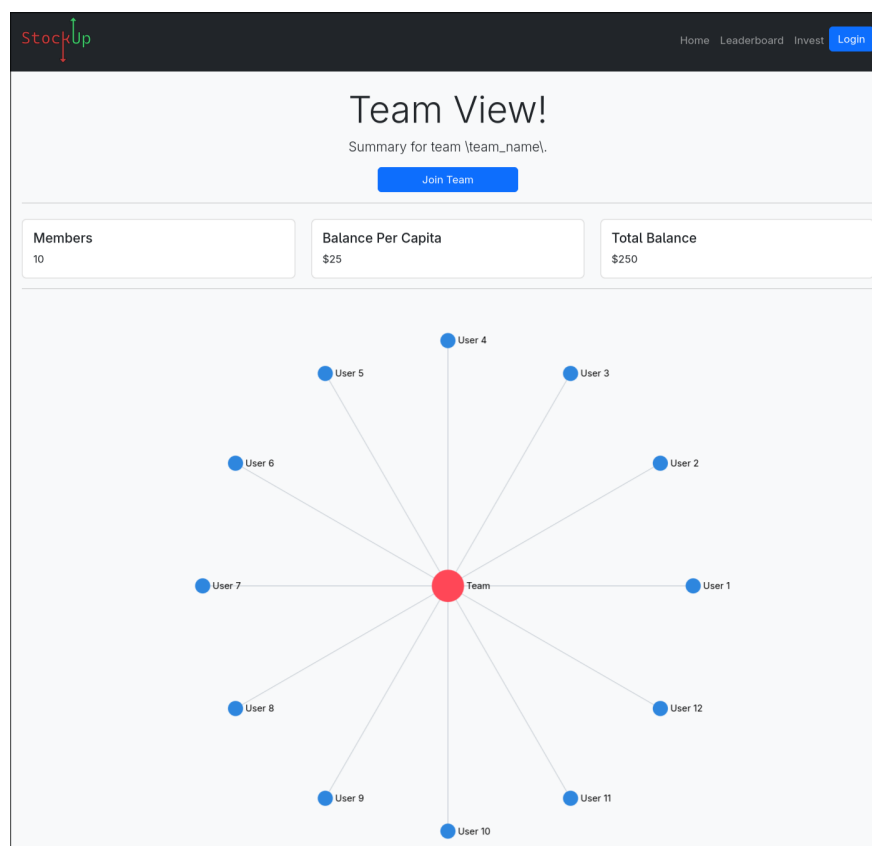


Figure 2: Co-Author Network

4.2 H-Index Filter

Our equivalent to the H-Index Filter is on the leaderboard page, where users can view the top performing teams and stocks. The H-Index Filter is defined to be "teams with a balance per capita greater than the mean balance per capita of all teams". These teams are shown in a graph.

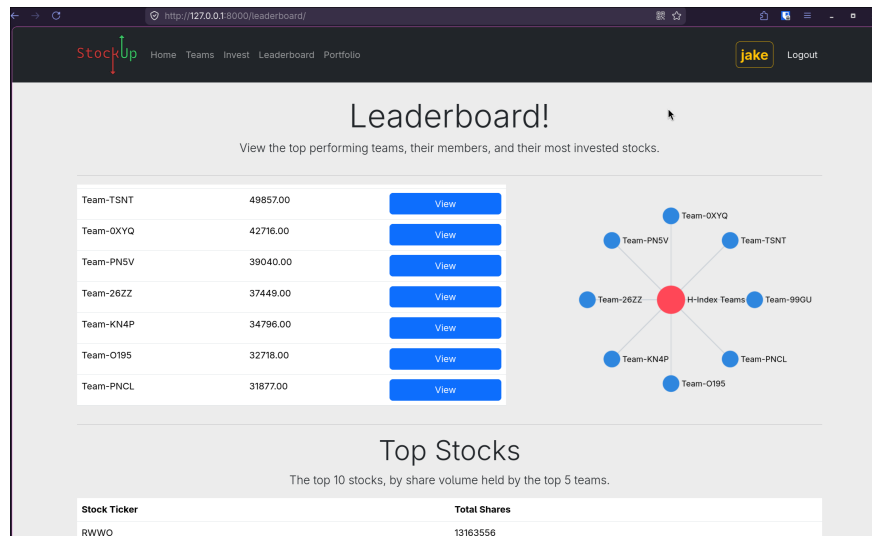


Figure 3: H-Index Filter

4.3 Q1 Journal Influence Network

Our equivalent to the Q1 Journal Influence Network is built upon the H-Index Filter. It gets the teams that are in the H-Index, gets the users of those teams, then gets the top 10 stocks, by volume, held by the users of the team. The user can zoom in any team node in the H-Index Filter Graph to view the Q1 Journal Influence Network Graph of that team.

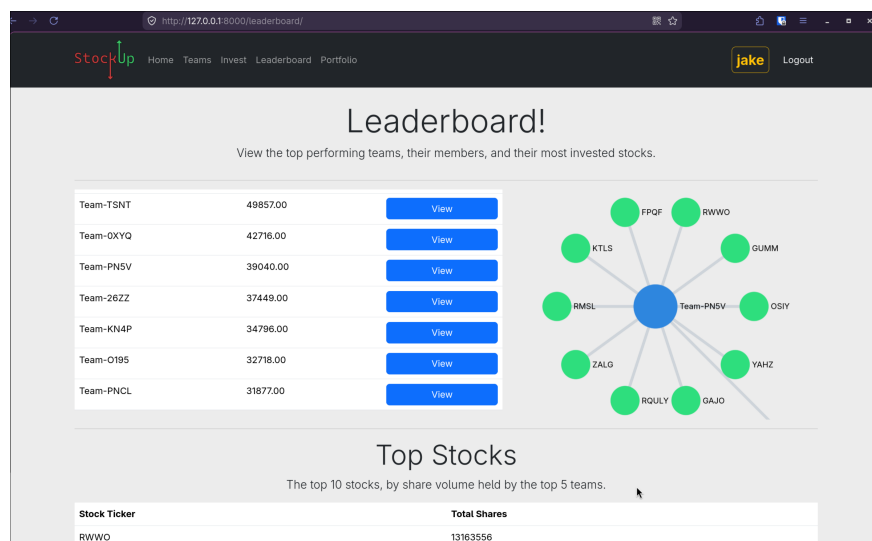


Figure 4: Q1 Journal Influence Network

5 Deviations From the Original Plan

While our project execution has remained largely as planned, there have been a few small things that have changed.

- Initially, the plan was that the leaderboard and team page would be the same. That turned out to be too much info for a single page.
- An attribute has been added to the team entity, which is an owner field. This allows the owner to delete the team if they made it.

6 Progress Reports

6.1 Phase I

- Frontend design done - currently hardcoded in for the presentation.
- Database implemented in Django ORM.
- Basic backend implemented, does not interact with database.

6.2 Phase II

- Backend and database integrated with frontend.
- Implemented Co-Author Network graph.
- Implemented portfolio page.

6.3 Phase III

- Reworked Leaderboard page.
- Implemented H-Index Filter and graph.
- Implemented Q1 Journal Influence Network and graph.
- Added login system and requirements to view pages.

6.4 Phase IV (Final Phase)

- Added the ability to sell stocks.
- Made creating a team automatically put you on the team.
- Small cosmetic changes to portfolio page.

7 Validation

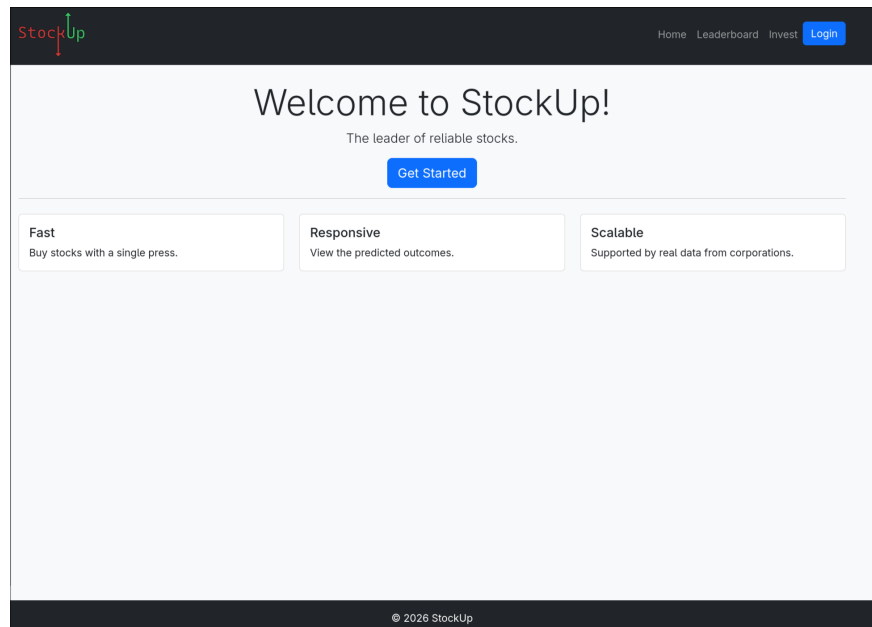


Figure 5: Home Page

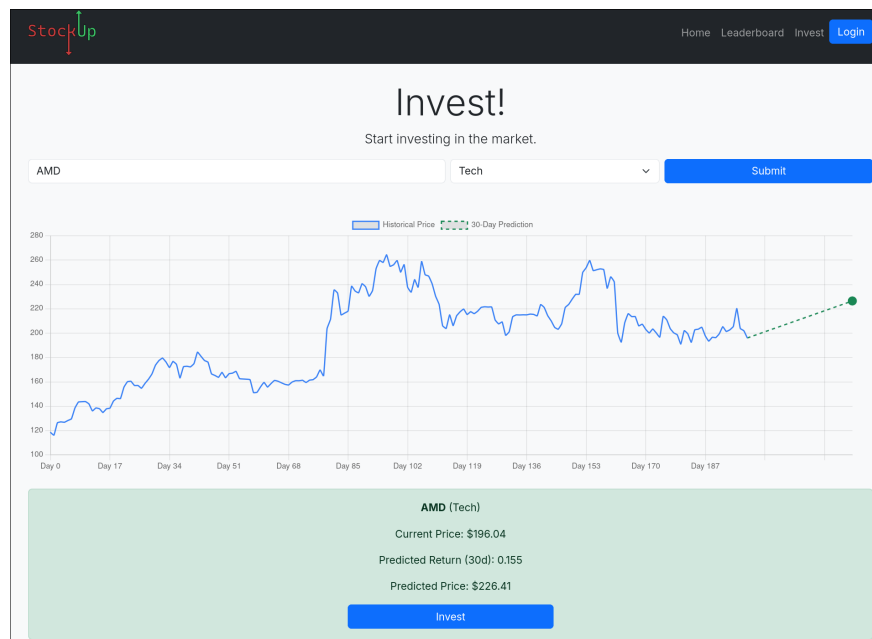


Figure 6: Stock Investing Page

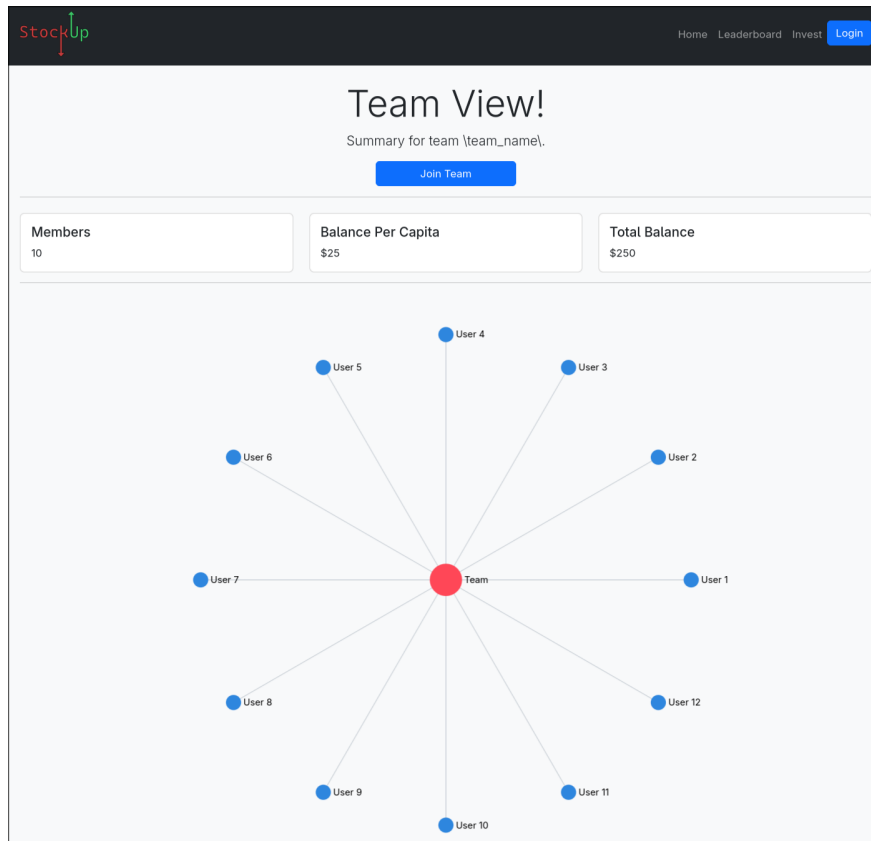


Figure 7: Team View Page

The screenshot shows the 'Login' page. The 'StockUp' logo and navigation links are at the top. The main heading is 'Login' with a link 'Don't have an account? [Sign up!](#)'. Below are two input fields: 'Email address' with a placeholder 'Enter email' and 'Password' with a placeholder 'Password'. A 'Submit' button is at the bottom. The footer contains the text '© 2026 StockUp'.

Figure 8: Login Page

StockUp

Home Leaderboard Invest [Login](#)

Sign Up

Already have an account? [Log in!](#)

Email address

Username

Password

Confirm Password

[Submit](#)

© 2026 StockUp

Figure 9: Sign Up Page

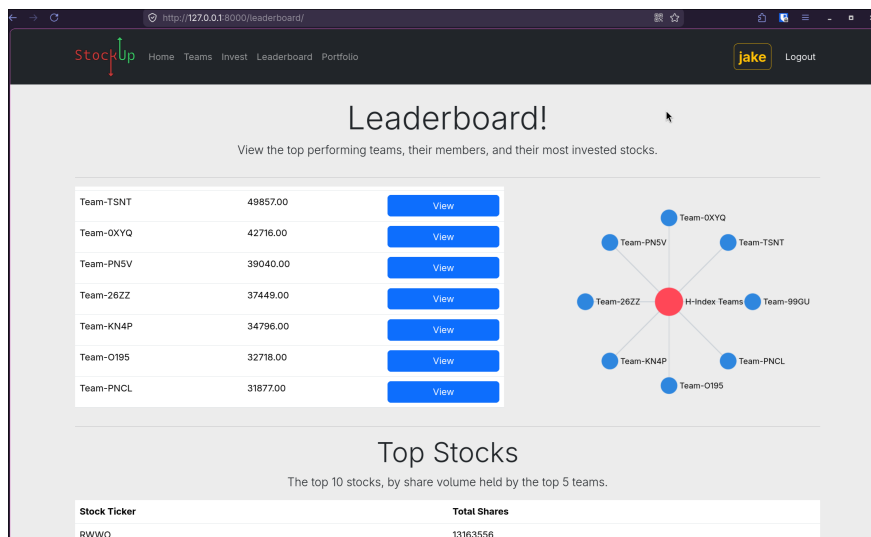


Figure 10: Leaderboards Page

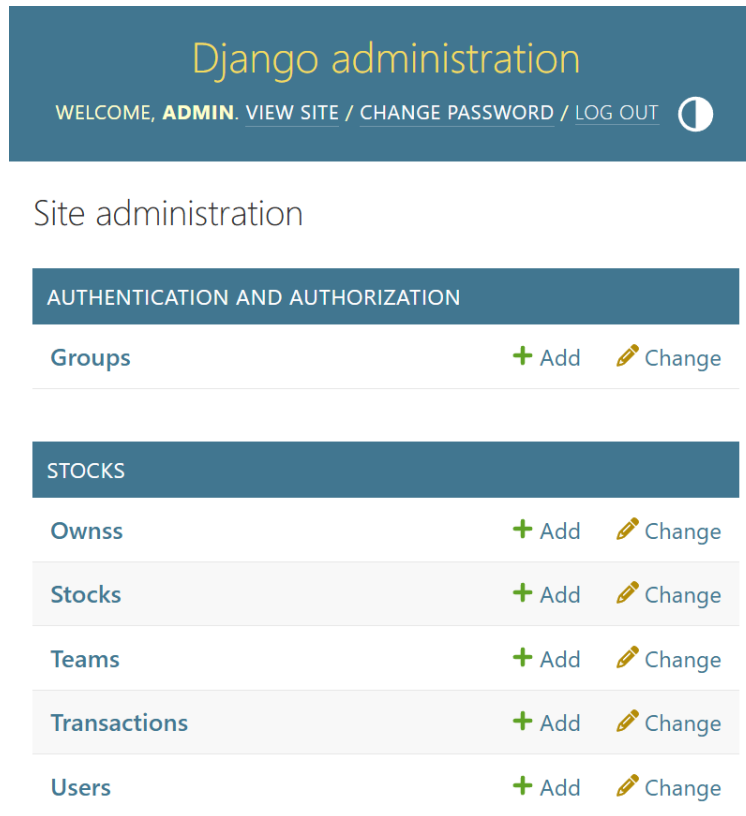


Figure 11: Database Administration Page

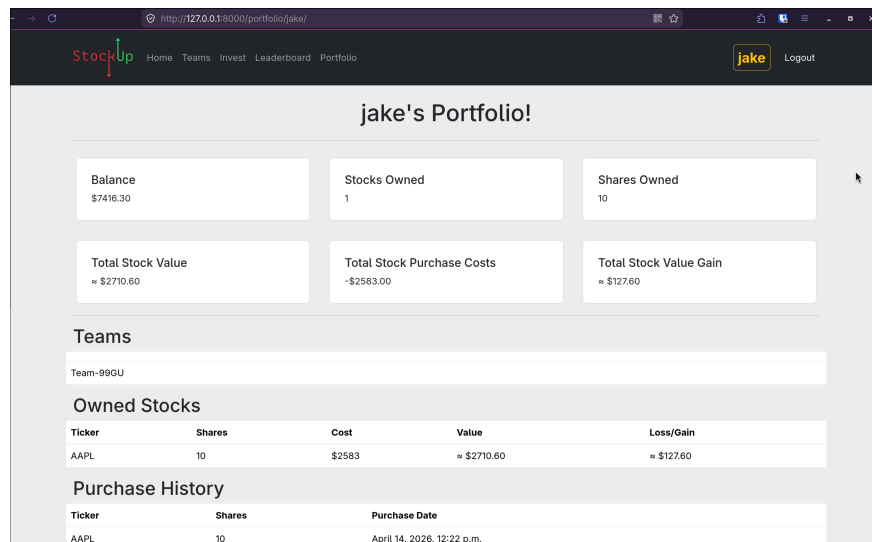


Figure 12: Portfolio Page

8 Timeline of Deliverables

- **April 15, 2026:** All parts of the project must work in isolation of each other. The website should be fully functional, the backend and API should be fully functional, and the database should be fully functional. At this point, the project should be done with Phase III.
- **May 1, 2026:** The project should be fully functional at this point. The required queries should be in place, and all aspects of the project should properly interact with minimal bugs. Phase IV should be complete at this point.
- **May 5, 2026:** The project must be complete by this point.

As of May 5, 2026, the project is complete and presented. All goals were achieved and the timeline was upheld.

9 License

This project is free, open-source software under the MIT license. This report and project are shareable without author consent for academic and research purposes. More information about the license can be found on our GitHub, which is at <https://github.com/mdcrapse/stock>.